Component classification and design rationale

This artifact groups the six required neural network components by their primary function. The organizing rule follows what each component contributes to a prediction or a training update. These are explanatory groups for this artifact, rather than a universal taxonomy.

Classification by primary function

Component	Functional group	Reason for placement
Layers	Architecture	Group units into input, hidden and output stages. Their order and size define the network's structure.
Neurons	Architecture	Provide the computational units within layers. Each non-input neuron combines weighted inputs and a bias, then applies an activation.
Weights	Learned parameters	Scale connections between units. Training adjusts them to change the model's predictions. Biases are additional learned parameters.
Activation functions	Signal transformation	Transform a neuron's weighted input. Nonlinear choices such as tanh enable nonlinear patterns; a linear activation passes the input through unchanged.
Loss functions	Training objective	Compare predictions with targets and assign an error value. The objective defines what training seeks to reduce.
Optimization algorithms	Parameter update	Use gradients to update weights and biases. Backpropagation calculates the gradients that the optimizer applies.

How the components were sorted

1. Choose a functional sorting rule. For each component, identify its main contribution to the forward computation or training cycle. This provides a consistent basis for grouping items that look different in a diagram.

2. Resolve overlapping roles. A neuron uses weights and an activation, but those parts retain distinct roles. Weights are learned values; an activation is the transformation applied to their weighted sum. Layers describe structure, rather than a set of trainable values by themselves.

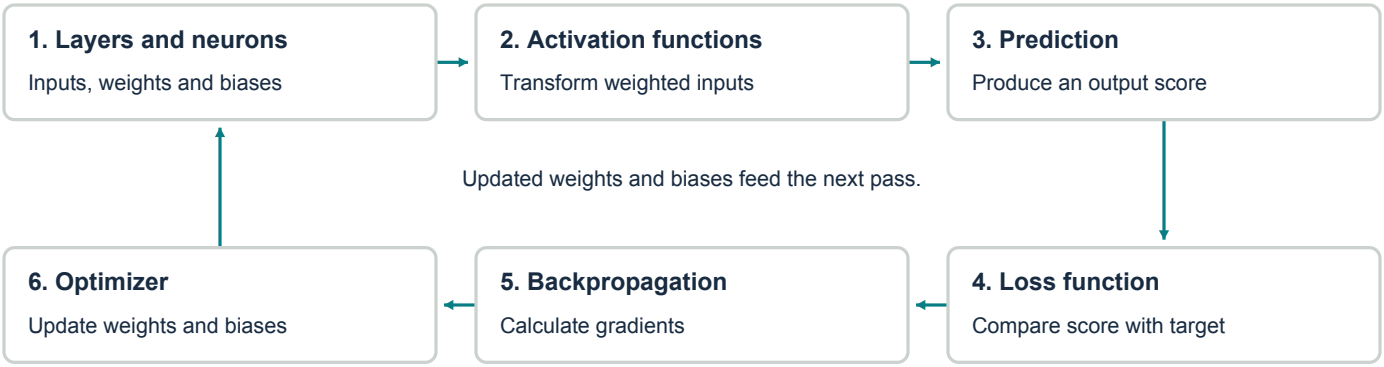
3. Separate the objective from the update rule. Loss evaluates a prediction against a target. The optimizer changes parameters to reduce that loss. Keeping these categories separate explains why backpropagation computes gradients while gradient descent applies updates.

4. Check the categories against equations and evidence. The worked neuron example connects the diagram to $z = \text{sum}(w_i x_i) + b$ and $a = f(z)$. Parameter counting confirms 18 weights plus 7 biases for the 2-4-2-1 network. Official Playground source confirms the demo's half-squared-error loss and classification output activation.

Definitions and parameter calculations: (Google, 2025b; TensorFlow, 2022b).

RELATIONSHIPS, EVIDENCE AND LEARNING

How the parts work together



During these runs, the optimizer changes learned weights and biases. Architecture and chosen activations remain fixed. Held-out test examples assess performance without driving parameter updates (TensorFlow, 2022b, 2022c).

Interpreting the recorded results

The same larger spiral model's test loss decreased from 0.451 at 200 epochs to 0.032 at 1,000 epochs. This is evidence of improvement with additional training in that run. Cross-configuration comparisons are exploratory because seeds or initialization differed, and the noisy circle stopped at 217 epochs. Values displayed as 0.000 are rounded. These loss values are not accuracy percentages, and no wall-clock training time was measured.

Reflection on learning

My main design decision was to organize the six neural network components by their primary function: architecture, learned parameters, signal transformation, the training objective, and parameter updates. This structure connects the definitions to a prediction and its training cycle. The worked neuron example makes the calculations inspectable, while the architecture diagram shows how signals pass between layers.

I revised the artifact to explain these choices more explicitly. I added a classification table with a reason for each placement and a separate training loop connecting predictions, loss, gradients, and updates. I also checked the parameter calculation: the 2-4-2-1 network contains 18 weights and seven biases, for 25 trainable parameters. These revisions make the relationships easier to follow and reduce ambiguity about what training changes.

The experiment record highlighted a challenge in drawing fair conclusions. Seeds differed across several configurations, and the noisy circle stopped at 217 epochs. I therefore placed these limitations beside the results. Continuing the same larger spiral model provided a clearer comparison: test loss decreased from 0.451 at 200 epochs to 0.032 at 1,000 epochs. This connects training effort to performance on held-out data without establishing that a larger network is always better. For future experiments, I would use matched epoch budgets and repeat each configuration across several seeds. In professional work, I would apply the same discipline by explaining model choices, showing the supporting evidence, and keeping conclusions within what that evidence demonstrates.

Full experiment record and original screenshots

References

Google. (2025a, August 25). *Neural networks: Activation functions*. https://developers.google.com/machine-learning/crash-course/neural-networks/activation-functions

Google. (2025b, December 3). *Neural networks: Nodes and hidden layers*. https://developers.google.com/machine-learning/crash-course/neural-networks/nodes-hidden-layers

Google. (2026, February 3). *Linear regression: Gradient descent*. https://developers.google.com/machine-learning/crash-course/linear-regression/gradient-descent

Smilkov, D., & Carter, S. (n.d.). *A neural network playground* [Computer software]. TensorFlow. Retrieved November 1, 2026, from https://playground.tensorflow.org/

TensorFlow. (2022a, June 6). *heatmap.ts* (Commit 02469bd) [Source code]. GitHub. https://github.com/tensorflow/playground/blob/02469bd3751764b20486015d4202b792af5362a6/src/heatmap.ts

TensorFlow. (2022b, June 6). *nn.ts* (Commit 02469bd) [Source code]. GitHub. https://github.com/tensorflow/playground/blob/02469bd3751764b20486015d4202b792af5362a6/src/nn.ts

TensorFlow. (2022c, June 6). *playground.ts* (Commit 02469bd) [Source code]. GitHub. https://github.com/tensorflow/playground/blob/02469bd3751764b20486015d4202b792af5362a6/src/playground.ts